ACH2118 - Introdução ao Processamento de Língua Natural

Universidade de São Paulo - Escola de Artes, Ciências e Humanidades
Professor: Ivandré Paraboni

Exercício Prático 2

Classificação de clareza de respostas do e-SIC

Grupo 3 - Relatório do projeto
Novembro de 2026

Integrante	Número USP
Felipe Teixeira Rodrigues	13672213
Gabriel Fernandes Marchesano	15453229
Gabriel Alexo Rocha	13729636
Nicolas Garcia Santos	15481919

Estratégia final: votação suave ponderada (*weighted soft voting*) de probabilidades calibradas, com 75% de contribuição do SVM lexical original e 25% do NorBERTo utilizado no EP1.

Resultado em Teste1: 447/900 acertos, acurácia de 49,67% e F1-macro de 49,58%. A comparação com a entrega efetiva do EP1 apresenta ganho de 15 acertos, sem superioridade estatisticamente significativa pelo teste de McNemar bilateral exato (p = 0,303327).

1. Estratégia adotada e relação com o EP1

O objetivo é classificar a clareza de respostas do e-SIC nas classes `c1`, `c234` e `c5`, correspondentes, respectivamente, à nota 1, ao agrupamento das notas 2 a 4 e à nota 5. No EP2, a estratégia escolhida combina duas representações complementares: TF-IDF de palavras e caracteres, classificado por um SVM linear, e representações contextuais do NorBERTo adaptado com LoRA no EP1. Após calibrar as distribuições dos dois modelos, a previsão é a classe de maior probabilidade na média ponderada com pesos 0,75 para SVM e 0,25 para NorBERTo.

O ponto de partida foi a conclusão do EP1: um modelo contextual moderno não havia demonstrado vantagem clara sobre referências lexicais muito mais leves. Em vez de pressupor que aumentar a complexidade resolveria o problema, investigamos se os erros de modelos diferentes poderiam se compensar. Essa hipótese motivou as tentativas com regressão logística (RL), Qwen, votação por maioria, stacking e classificação assistida por GPT, incluindo recuperação de exemplos por RAG.

O percurso produziu ganhos pontuais, mas também mostrou que corrigir alguns erros não basta: uma intervenção pode simultaneamente prejudicar casos antes corretos. Por esse motivo, além das métricas agregadas, acompanhamos correções e prejuízos pareados, desempenho por classe, separação dos dados e custo computacional. A decisão final foi manter uma composição pequena, já selecionada no desenvolvimento, que não exige chamadas a um modelo remoto.

O componente NorBERTo é o adaptador original da entrega do EP1, preservado como referência e incorporado à solução final sem alterações em seus pesos.

2. Dados, partições e critérios de avaliação

2.1. Dados disponíveis

Utilizamos a mesma base de treinamento do EP1, com 20.092 respostas: 6.347 exemplos `c1`, 6.853 `c234` e 6.892 `c5`. A entrada de classificação é exclusivamente a coluna `resp_text`. A pergunta original do cidadão e o conteúdo dos anexos mencionados nas respostas não estão disponíveis, o que limita a observação do contexto necessário para avaliar a clareza.

Conjunto ou papel	Respostas	Grupos de similaridade	Finalidade
Treinamento dos modelos-base	16.093	12.956	Ajuste dos classificadores
Calibração	2.027	1.627	Ajuste das escalas de probabilidade
Meta-validação	1.972	1.654	Pesos da votação suave e meta-modelo
Teste1	900	877 textos normalizados distintos	Comparação retrospectiva com o EP1
Teste2	897	-	Avaliação oficial do EP2

As duas partes de 2.027 e 1.972 respostas resultam da divisão das 3.999 respostas reservadas no EP1. Seus grupos são componentes de similaridade textual, enquanto a contagem de Teste1 se refere a textos normalizados distintos; as duas medidas de agrupamento não são equivalentes. Na

calibração há 635/710/682 exemplos por classe; na meta-validação, 622/707/643. Teste1 tem 300 exemplos de cada classe.

2.2. Separação por grupos

Preservamos a divisão original de treinamento e conjunto reservado. Ela agrupa respostas próximas com normalização de identificadores e similaridade de TF-IDF de caracteres, usando limiar de cosseno 0,93. A subdivisão do conjunto reservado utiliza `StratifiedGroupKFold`, com duas partes e semente 42, mantendo cada componente de similaridade em um único papel. Não há compartilhamento desses grupos entre treinamento, calibração e meta-validação.

2.3. Métricas e alcance das comparações

Reportamos a acurácia, solicitada no enunciado, e o F1-macro, adotado como objetivo principal nas seleções do EP2 para atribuir o mesmo peso às três classes. Acurácia é a fração de previsões corretas; F1-macro é a média dos F1 de cada classe. Diferenças percentuais entre métodos são apresentadas em pontos percentuais (pp).

Resultados de seleção na meta-validação, previsões fora do fold (*out-of-fold*, OOF), pilotos difíceis e Teste1 são identificados separadamente. Eles avaliam populações e procedimentos diferentes. Não tratamos seus valores absolutos como uma única classificação de desempenho.

3. Pré-processamento e representações

3.1. Classificadores lexicais

No SVM e na RL, entidades HTML são convertidas para caracteres, sequências de espaços são reduzidas a um espaço e o texto é convertido para minúsculas. Preservam-se acentos e negações; não há remoção manual de pontuação, lematização, stemming ou lista de palavras de parada. O analisador de palavras segue sua tokenização própria, e o ramo de caracteres também representa padrões de pontuação presentes no texto.

A representação original concatena TF-IDF de unigramas e bigramas de palavras com n-gramas `char_wb` de 3 a 5 caracteres. O ramo de palavras tem `min_df=2`, `max_df=0.98` e até 60.000 atributos; o de caracteres tem `min_df=3`, `max_df=1.0` e até 50.000. Ambos usam frequência sublinear, IDF suavizado, normalização L2 e valores `float32`. Os ramos têm o mesmo peso.

Vocabulário e IDF são aprendidos apenas no treinamento correspondente. Na validação cruzada da busca final do SVM, cada fold ajusta sua própria representação com a parte de treino, impedindo que a validação influencie a extração de atributos.

3.2. Modelos contextuais e GPT

O NorBERTo preserva maiúsculas e minúsculas, converte entidades HTML e normaliza espaços. Utiliza seu tokenizador original e no máximo 512 tokens, com truncamento que preserva o

início e o fim da resposta. Não alteramos textos nem rótulos para resolver inconsistências aparentes de anotação.

O Qwen3-1.7B foi adaptado à classificação das três classes com QLoRA e contexto de 256 tokens. Utilizou a mesma divisão de 16.093/3.999 respostas, embora as 3.999 tenham participado da escolha de seu checkpoint. Essa exposição herdada limita a independência de sua avaliação posterior no ensemble.

Nas experiências com GPT, os alvos preservam o texto original. O rótulo verdadeiro do alvo nunca compõe o pedido ao modelo. Quando há demonstrações, os exemplos rotulados vêm do treinamento e são identificados separadamente do texto a classificar. RAG recupera exemplos de respostas e suas classes; não recupera perguntas ou anexos ausentes da base.

4. Trajetória experimental

4.1. SVM e regressão logística como referências fortes

Os classificadores lexicais permaneceram competitivos. Na comparação comum de Teste1, o SVM original obteve 445/900 acertos (49,44%) e a RL de desenvolvimento, 447/900 (49,67%), contra 432/900 (48,00%) do NorBERTo entregue no EP1. Esses números motivaram o estudo de combinação: havia pouco espaço para ganho apenas pela substituição de um classificador já forte.

SVM e RL usam atributos semelhantes e concordaram em 861 das 900 previsões (95,67%). Portanto, sua presença simultânea em uma votação não representa duas fontes independentes de evidência. A elevada concordância ajuda a interpretar por que incluir mais modelos lexicais não trouxe automaticamente melhor resultado.

4.2. Votação por maioria

Testamos 15 configurações entre modelos isolados, pares e o trio NorBERTo/SVM/RL, com diferentes árbitros para empates. Elas produziram somente seis sequências de previsões distintas. O melhor trio observado utilizou RL nos empates de três classes e acertou 443/900 respostas, com acurácia de 49,22% e F1-macro de 48,86%, abaixo da RL isolada.

Os três modelos concordaram em 579 casos; em 316 houve maioria de dois contra um; apenas cinco apresentaram três rótulos distintos. Como SVM e RL quase sempre concordavam, a maioria frequentemente repetia a decisão lexical. Também identificamos combinações de dois modelos cujo desempate por um dos próprios votantes reproduzia exatamente esse votante. Esses casos não constituem uma melhoria de ensemble.

Como Teste1 já havia sido consultado, essa comparação tem caráter exploratório. Os resultados ajudam a caracterizar a diversidade dos votos, mas não constituem uma seleção confirmada em teste independente.

4.3. Qwen: maior capacidade sem ganho na combinação

Adaptamos o Qwen3-1.7B com quantização NF4, LoRA de posto 8 e alpha 16, lote 4, acumulação de gradiente 4 e três épocas. O treinamento realizou 3.018 atualizações e levou aproximadamente 3 h 8 min na GPU AMD Radeon RX 6650 XT. O checkpoint selecionado obteve 1.791/3.999 acertos na validação (44,79%) e 414/900 em Teste1 (46,00%; F1-macro de 45,53%).

O modelo foi incorporado tanto à votação suave quanto ao stacking. Na votação suave, a grade de quatro candidatos selecionou SVM 75%, RL 0%, NorBERTo 25% e Qwen 0%, reproduzindo os mesmos 900 rótulos da combinação sem Qwen. No stacking, suas probabilidades receberam coeficientes não nulos, mas o total de acertos caiu de 442 para 440. A presença de um modelo maior e treinado para a tarefa não gerou contribuição adicional suficiente nas combinações avaliadas.

4.4. Stacking de probabilidades

O stacking substituiu os pesos globais fixos por uma regressão logística multinomial capaz de aprender combinações diferentes por classe. A versão de três modelos recebeu nove atributos: as três probabilidades calibradas de SVM, RL e NorBERTo. A versão com Qwen recebeu doze. Somente o meta-modelo foi ajustado nas 1.972 respostas; os modelos-base e calibradores permaneceram congelados.

Comparamos `C` em {0,01; 0,1; 1; 10}, em cinco folds estratificados por grupos. A regra de um erro-padrão escolheu `C=0.1`, favorecendo maior regularização entre resultados próximos. A meta-RL utiliza penalização L2, intercepto, solver `lbfgs`, sem pesos de classe ou escala adicional dos atributos.

Em Teste1, stacking de três modelos obteve 442/900 acertos, acurácia de 49,11% e F1-macro de 49,03%; com quatro, 440/900, 48,89% e 48,80%. A versão de três corrigiu 40 erros da votação suave, mas prejudicou 45 acertos. Seu ganho em `c234` não compensou as perdas em `c1` e `c5`. O resultado não comprova que stacking seja inferior em geral; mostra que a flexibilidade adicional não trouxe ganho neste protocolo.

4.5. GPT como classificador e como árbitro seletivo

A primeira experiência completa com GPT/Codex, usando exemplos de treino recuperados, acertou 382/900 respostas (42,44%; F1-macro de 40,39%). A principal dificuldade foi `c1`: apenas 56 de seus 300 exemplos foram identificados corretamente, revocação de 18,67%. O método favoreceu respostas de alta clareza e perdeu 50 acertos líquidos frente ao EP1.

Investigamos então uma hipótese mais restrita: usar GPT somente quando os classificadores locais estivessem incertos ou discordassem. O roteador combinou entropia, probabilidade máxima, margem entre as duas classes mais prováveis e divergência entre as previsões nativas. Seu desenvolvimento utilizou previsões OOF agrupadas aninhadas, com cinco folds externos e cinco internos para o stacking, evitando usar o próprio grupo-alvo no ajuste do meta-modelo ou na escolha interna de `C`.

Comparamos prompts com apenas o texto e com texto acrescido de previsões locais, apresentadas explicitamente como falíveis. Os pedidos usaram `gpt-6.1-sol`, raciocínio baixo, saída JSON validada, lotes de até oito alvos, duas solicitações simultâneas e cache persistente. Esse nome identifica o modelo solicitado ao serviço; não é uma revisão imutável que assegure decisões idênticas em futuras chamadas.

Orçamento de alvos enviados ao GPT	Acertos OOF / 1.972	F1-macro OOF	Acertos Teste1 / 900	F1-macro Teste1
0%: stacking local	936	47,30%	440	48,80%
5%	937	47,28%	435	48,18%
10%	929	46,84%	434	48,05%
20%	937	47,10%	436	48,18%

Na validação, os orçamentos de 5% e 20% ganharam um acerto cada, mas reduziram o F1-macro, métrica principal. Assim, o orçamento selecionado foi zero antes da avaliação dessa rodada em Teste1. Nos três orçamentos positivos, o GPT também reduziu o resultado observado em Teste1. A cascata realizou 97 chamadas no conjunto das etapas, sem falhas finais; isso não justificou sua adoção.

4.6. RAG: exemplos semelhantes não garantem a mesma clareza

Para investigar se o problema estava no contexto fornecido ao GPT, comparamos recuperação lexical por TF-IDF, semântica por embeddings E5 e híbrida por fusão de posições recíprocas. O banco partiu somente das 16.093 respostas de treinamento; após excluir exemplos vazios, longos demais, duplicados ou com rótulos conflitantes, continha 11.562 demonstrações elegíveis. As 3.999 respostas reservadas ficaram fora dele.

A recuperação também excluiu duplicatas e quase duplicatas do alvo e diversificou os exemplos por relevância e redundância. O piloto avaliou 14 combinações de contexto nas mesmas 48 respostas OOF difíceis, com três ou cinco demonstrações e presença ou ausência das previsões locais. O melhor braço, híbrido com três exemplos e previsões locais, obteve 19/48 acertos, contra 16/48 do stacking. Corrigiu oito erros e prejudicou cinco; o ganho exploratório em amostra pequena não foi significativo (McNemar exato p = 0,5811).

Na confirmação, excluímos todos os grupos do piloto, restando 1.884 respostas. A receita híbrida de três exemplos foi mantida, e comparamos texto, previsões locais, RAG e RAG com previsões locais nos orçamentos de 0%, 5% e 10%.

Contexto da confirmação	Roteamento GPT	Acertos / 1.884	Acurácia	F1-macro
Stacking local	0%	910	48,30%	48,08%
Apenas previsões locais	5%	906	48,09%	47,84%
RAG híbrido + previsões locais	5%	906	48,09%	47,83%
RAG híbrido + previsões locais	10%	907	48,14%	47,82%

Nenhuma intervenção positiva superou o stacking nessa confirmação. Com RAG e previsões locais a 10%, houve 29 correções e 32 prejuízos; 26 dos prejuízos ocorreram em `c234`. O ganho do piloto não persistiu e o orçamento selecionado voltou a ser zero. Piloto e confirmação somaram 178 chamadas válidas e três tentativas falhas arquivadas, sem novas chamadas na avaliação dos resultados.

Uma investigação posterior, inteiramente offline, avaliou 171 variantes de recuperação e duas referências de controle, além de políticas de roteamento e aceitação sobre respostas GPT já registradas. A configuração de recuperação escolhida em uma parte do desenvolvimento não demonstrou ganho na auditoria por grupos. O seletor conjunto de intervenção obteve F1-macro de 47,19% nas 1.972 respostas, contra 47,30% do stacking, com intervalo descritivo de 95% da diferença entre -0,71 e +0,45 pp.

Essa reavaliação tinha respostas GPT observadas para somente 236 alvos e conservava previsões históricas fixas dos estágios anteriores. Portanto, não constitui uma nova avaliação aninhada de ponta a ponta. A conclusão prática foi encerrar essa linha sem novas chamadas: textos semelhantes em assunto e forma podem carregar notas distintas, e a similaridade de recuperação não é uma medida direta de clareza.

5. Busca de hiperparâmetros e controle de custo

5.1. Espaços avaliados

A tabela resume os principais parâmetros investigados no EP2. Para o SVM, os desenhos são fracionários: as opções de representação foram agrupadas em receitas predefinidas, e não combinadas em todo produto cartesiano possível.

Etapa	Parâmetro	Valores ou alternativas avaliados
Calibração original	SVM / modelos probabilísticos	Sigmoide / identidade ou temperatura em [0,25; 10]
Calibração do novo SVM	Método	Sigmoide ou isotônica, com seleção aninhada
Votação suave, três modelos	Pesos não negativos, soma 1	15 pontos em passos de 0,25 + pesos iguais: 16 misturas
Votação suave, quatro modelos	Pesos não negativos, soma 1	35 pontos distintos em passos de 0,25
Stacking	Regularização `c`	{0,01; 0,1; 1; 10}
GPT seletivo	Orçamento de alvos	{0%; 5%; 10%; 20%}
RAG	Recuperador / demonstrações	Lexical, semântico ou híbrido / {3; 5}
Confirmação RAG	Contexto / orçamento	Texto, local, RAG ou ambos / {0%; 5%; 10%}

Os parâmetros do Qwen descritos na seção 4.3 constituem a configuração treinada, sem alegação de uma busca exaustiva sobre todos eles. De modo semelhante, `C=0.25` da RL foi mantido da referência de desenvolvimento; não foi escolhido com os rótulos de Teste1.

A otimização final do SVM concluiu duas fases de validação cruzada agrupada: 96 configurações em três folds na fase A e 20 configurações em cinco folds na fase B. A segunda fase refinou duas receitas de palavras + `char_wb`: a sublinear original e uma binária. Os planos e

resultados estão em `results/svm-final-budgeted-20261106/phase-a/` e `phase-b/`, com agregados da fase A e da fase B. A tabela apresenta os valores efetivamente avaliados, respeitando as receitas fracionárias, sem implicar um produto cartesiano entre todas as opções.

Parâmetro do SVM / TF-IDF	Fase A	Fase B
Representação	Palavras, `char`, `char_wb`, palavras + `char_wb`	Palavras + `char_wb`
Regularização `C`	{0.005; 0.05; 0.5}	{0.0158113883008419; 0.05; 0.158113883008419}
`class_weight`	{`None`; `balanced`}	{`None`; `balanced`}
Perda `loss`	`squared_hinge`	{`squared_hinge`; `hinge`}
`ngram_range` de palavras	{(1,1); (1,2); (1,3)}	(1,2)
`ngram_range` de caracteres	{(2,4); (3,5); (3,6)}	{(3,5); (3,6)}
`min_df` de palavras	{1; 2; 5}	{2; 5}
`min_df` de caracteres	{2; 3}	3
`max_df` de palavras	{0.98; 1.0}	0.98
`max_df` de caracteres	{0.95; 0.98; 1.0}	{0.95; 1.0}
Pares (`binary`, `sublinear_tf`) dos ramos	{(`False`, `True`); (`False`, `False`); (`True`, `False`)}	{(`False`, `True`); (`True`, `False`)}
Normalização `norm`	{`l1`; `l2`}	`l2`
Peso do ramo de caracteres, com peso de palavras fixo em 1	1.0	{0.5; 1.0; 2.0}
`max_features` (fixo por ramo)	Palavras: 60.000; caracteres: 50.000	Palavras: 60.000; caracteres: 50.000

Nas duas fases, `penalty='l2'`, `tol=1e-4` e `max_iter=20000` permaneceram fixos; `dual='auto'` foi usado com `squared_hinge` e `dual=True` com `hinge`. As sementes foram 42 na fase A e 143 na fase B; o reajuste final da configuração selecionada usou semente 42. Esses controles não foram objeto de busca.

Melhores valores encontrados e configuração mantida. A maior média de F1-macro da fase B foi 44,1344%, mas a regra predefinida de um erro-padrão selecionou o candidato `42c22ab6e0398e33b3ea`, com média de 43,8293%, dentro do limiar de 43,7369% e com menos atributos. A tabela distingue o máximo observado, a configuração selecionada para o novo SVM e a configuração original efetivamente mantida na entrega, conforme o registro da otimização final.

Parâmetro	Maior média na fase B	Selecionado pela regra de um erro-padrão	Original mantido
Representação	Palavras + `char_wb`	Palavras + `char_wb`	Palavras + `char_wb`
`C`	0.05	0.05	0.05
`class_weight`	`balanced`	`None`	`None`
`loss`	`squared_hinge`	`squared_hinge`	`squared_hinge`
`ngram_range` de palavras	(1,2)	(1,2)	(1,2)

Parâmetro	Maior média na fase B	Selecionado pela regra de um erro-padrão	Original mantido
`ngram_range` de caracteres	(3,5)	(3,6)	(3,5)
`min_df` de palavras	2	5	2
`min_df` de caracteres	3	3	3
`max_df` de palavras	0.98	0.98	0.98
`max_df` de caracteres	1.0	0.95	1.0
`binary` (ambos os ramos)	`False`	`True`	`False`
`sublinear_tf` (ambos os ramos)	`True`	`False`	`True`
`norm` (ambos os ramos)	`l2`	`l2`	`l2`
Peso de palavras / caracteres	1.0 / 0.5	1.0 / 1.0	1.0 / 1.0
`max_features` de palavras / caracteres	60.000 / 50.000	60.000 / 50.000	60.000 / 50.000

O novo SVM não foi adotado porque seu ensemble obteve apenas +0,0164 pp de F1-macro na meta-validação, melhorou em dois dos cinco folds e apresentou intervalo pareado de 95% que inclui zero, falhando nos critérios predefinidos de substituição. A decisão, congelada antes de Teste1, manteve o SVM original com peso 0,75 e o NorBERTo original com peso 0,25.

6. Método final e procedimento de reprodução experimental

6.1. Calibração e composição

O `LinearSVC` não fornece probabilidades nativas. Suas margens recebem calibração sigmoide um-contra-restante, seguida de normalização multiclasse, nas 2.027 respostas destinadas à calibração. O estimador-base permanece congelado. Não misturamos margens brutas do SVM diretamente com probabilidades do modelo neural.

Para o NorBERTo, uma verificação agrupada de três folds na mesma parte de calibração comparou a distribuição original com uma temperatura escalar. O valor final foi T = 1,1171232074. Temperatura positiva preserva a classe individual de maior probabilidade, mas modifica sua confiança relativa ao combinar modelos. A RL e o Qwen também foram calibrados durante os experimentos, embora tenham recebido peso zero na mistura escolhida.

A votação suave compara as misturas pelo F1-macro nas 1.972 respostas de meta-validação; em empate, usa acurácia e proximidade dos pesos uniformes. O resultado final é:

```
p_final(c | x) = 0.75 * p_SVM_calibrado(c | x)
               + 0.25 * p_NorBERTo_calibrado(c | x)
prediction(x) = argmax_c p_final(c | x)
```

Componente	Configuração final mantida
SVM original	LinearSVC, `C=0.05`, L2, `squared_hinge`, sem peso de classe, semente 42
TF-IDF de palavras	(1,2), `min_df=2`, `max_df=0.98`, até 60.000 atributos
TF-IDF de caracteres	`char_wb` (3,5), `min_df=3`, `max_df=1.0`, até 50.000 atributos

Componente	Configuração final mantida
Frequência e escala dos ramos	TF sublinear, IDF suavizado, L2, pesos iguais
NorBERTo original do EP1	LoRA posto 4, alpha 8, dropout 0,05; duas épocas; contexto 512, início/fim
Calibração	SVM sigmoide; NorBERTo com temperatura 1,1171232074
Pesos finais	SVM 0,75; NorBERTo 0,25; RL e Qwen 0

6.2. Encadeamento das etapas

O procedimento preserva os modelos treinados nas 16.093 respostas, calibra suas saídas nas 2.027 respostas e escolhe a combinação nas 1.972 restantes. Em seguida, congela parâmetros e identidades dos arquivos. A predição recebe somente textos, verifica a ordem das classes `c1`, `c234`, `c5` e persiste rótulos, probabilidades, índices e hashes dos textos antes da avaliação separada.

Os pacotes de três e quatro candidatos conservam a mesma composição efetiva. Para executar especificamente a solução de dois contribuintes, indicamos explicitamente o artefato original de votação suave, evitando a necessidade de executar Qwen, cujo peso é zero. Essa escolha de artefato não refaz seleção ou treinamento.

7. Resultados e comparação com a entrega do EP1

7.1. Comparação comum em Teste1

A tabela mantém as 900 respostas e suas posições originais. Os modelos individuais são avaliados por suas decisões nativas; os ensembles usam as distribuições calibradas. Os valores são retrospectivos, pois Teste1 já havia sido consultado no histórico do projeto.

Método	Acertos / 900	Acurácia	F1-macro
GPT/Codex inicial	382	42,44%	40,39%
Qwen3-1.7B selecionado	414	46,00%	45,53%
NorBERTo entregue no EP1	432	48,00%	47,86%
SVM original	445	49,44%	49,06%
RL de desenvolvimento	447	49,67%	49,32%
Maioria de três modelos, desempate RL	443	49,22%	48,86%
Stacking de três modelos	442	49,11%	49,03%
Stacking de quatro modelos	440	48,89%	48,80%
Soft Voting original 75/25 - escolhido	**447**	**49,67%**	**49,58%**
Soft Voting com novo SVM - não adotado	446	49,56%	49,52%

A solução escolhida empata com a RL em acurácia e apresenta F1-macro 0,26 pp maior. Supera numericamente o SVM original por dois acertos e o NorBERTo por quinze. Essas diferenças pequenas não estabelecem superioridade estatística. A grade que incluiu Qwen repetiu os mesmos 447 acertos porque lhe atribuiu peso zero.

O baseline de classe majoritária selecionado no treino prevê `c5` e obtém 33,33% em Teste1. Trata-se de uma referência elementar; ela é distinta do baseline oficial de regressão logística mencionado no enunciado.

7.2. Desempenho por classe

Classe	Precisão	Revocação	F1	Exemplos
`c1`	52,85%	46,33%	49,38%	300
`c234`	44,26%	45,00%	44,63%	300
`c5`	52,11%	57,67%	54,75%	300
Média macro	49,74%	49,67%	49,58%	900

A matriz de confusão apresenta classes verdadeiras nas linhas e previsões nas colunas:

Verdadeira / prevista	`c1`	`c234`	`c5`
`c1`	139	93	68
`c234`	74	135	91
`c5`	50	77	173

O ensemble mantém maior revocação em `c5`, enquanto a classe intermediária tem o menor F1. Em relação ao EP1, os acertos de `c1` aumentam de 118 para 139 e os de `c5`, de 159 para 173; em `c234`, diminuem de 155 para 135. Portanto, o ganho global não é uniforme entre as classes.

7.3. Teste de McNemar contra os rótulos efetivamente entregues

A comparação utilizou a planilha entregue no EP1, `EP1/deliveries/test1.xlsx`, e não uma versão retreinada do modelo. Os textos e posições coincidem com as 900 respostas avaliadas, e os rótulos entregues coincidem integralmente com os do adaptador original preservado.

Resultado pareado	Quantidade
EP1 e EP2 acertam	347
Apenas EP1 acerta	85
Apenas EP2 acerta	100
EP1 e EP2 erram	368

Para os discordantes b = 85 e c = 100, a estatística qui-quadrado com correção de continuidade é `(abs(b-c)-1)^2 / (b+c) = 1,059459`, com p assintótico de 0,303339. O teste bilateral exato fornece **p = 0,303327**, levando à mesma conclusão.

Acurácia passa de 48,00% para 49,67% (+1,67 pp) e F1-macro de 47,86% para 49,58% (+1,72 pp). Entretanto, **não rejeitamos a hipótese de igualdade das probabilidades de erro a 5%**. Não há evidência suficiente de superioridade do EP2 sobre o EP1 pelo critério solicitado. Isso não demonstra equivalência entre os métodos. McNemar compara acertos pareados, não testa diretamente a diferença de F1-macro.

8. Interpretação global e local

Utilizamos a RL de desenvolvimento já treinada como classificador simples interpretável, sem novo ajuste. Para cada classe, seu escore é a soma do intercepto com os produtos entre atributos TF-IDF e respectivos coeficientes. Essa decomposição é exata para o logit da RL; não explica diretamente o mecanismo interno do NorBERTo, do GPT ou a decisão completa da votação suave.

8.1. Características globais

A tabela apresenta exemplos de coeficientes positivos entre os atributos de palavras da RL. O modelo também inclui caracteres; a seleção abaixo serve à leitura linguística e não afirma que palavras sejam seus únicos atributos relevantes.

Classe	Atributo	Coeficiente
`c1`	`demanda`	+0,3498
`c1`	`para atendimento`	+0,3414
`c1`	`solicitação`	+0,3099
`c234`	`sistemas`	+0,3698
`c234`	`requisitos`	+0,3588
`c234`	`segue anexa`	+0,3548
`c5`	`dnpm`	+0,6728
`c5`	`questionário`	+0,6189
`c5`	`solicitação encaminhamos`	+0,5443

Termos como `dnpm` e `sistemas`, além de nomes próprios presentes entre os maiores coeficientes, indicam que a classificação pode explorar associações com órgão, assunto e modelos administrativos. São padrões estatísticos da base, sem garantia de representar critérios gerais de clareza. A ocorrência de `segue anexa` como indício de `c234`, por exemplo, não permite concluir que toda resposta com anexo pertença a essa classe.

8.2. Decomposições locais

Os índices a seguir são linhas reais do Excel, contando o cabeçalho na linha 1. Eles foram selecionados retrospectivamente para análise, sem mudança nas previsões ou nos parâmetros.

Linha 77: “Prezado Senhor.” O gabarito é `c5`; tanto RL quanto votação suave preveem `c1`. Na RL, o bigrama `prezado senhor` contribui +0,11058 para o logit de `c1`, e `senhor`, +0,08537. Somando todos os atributos de palavras e caracteres (+0,25704) ao intercepto (+0,00890), reconstruímos o escore de `c1`, +0,26594. O modelo reconhece uma fórmula textual frequente, embora ela quase não contenha informação substantiva.

Linha 693: “Sua resposta segue em anexo.” O gabarito é `c1`; RL e votação suave preveem `c5`. Para o logit de `c5` da RL, `anexo` contribui +0,06057 e `em anexo`, +0,04747, enquanto `segue em` contribui -0,08901. A soma de todos os atributos é +0,24568 e o intercepto, +0,15937, reconstruindo

o escore +0,40505. Há evidências lexicais em direções diferentes; o escore individual não equivale a uma justificativa humana de clareza nem a uma probabilidade calibrada do ensemble.

As extrações globais e quatro decomposições locais completas estão em `docs/report/data/interpretability.json`, com hashes do modelo e das previsões. O script `scripts/report/explain.py` verifica alinhamento dos textos e reconstrução dos escores, sem treinar modelo ou realizar chamada remota.

9. Análise manual de erros e limitações

9.1. Exemplos incorretamente classificados

Conteúdo inacessível por referência a anexo. Na linha 693, o texto informa somente que a resposta segue em anexo, mas o arquivo não está disponível. Na linha 137, “Prezada cidadã, Segue anexa a resposta. Atenciosamente,”, o gabarito também é `c1` e a votação suave prevê `c5`. A presença de uma fórmula administrativa fluente pode se associar a alta clareza no corpus, embora não revele o conteúdo efetivamente enviado. Sem anexo e pergunta, a análise não consegue determinar o motivo exato da nota humana.

Saudação sem conteúdo substantivo. A linha 77, rotulada como `c5`, mostra o problema inverso: o texto visível é uma saudação, mas o rótulo é de clareza máxima. Não corrigimos essa anotação por interpretação própria. O caso evidencia a dificuldade de transferir uma definição intuitiva de clareza para o padrão de anotação observado.

Resposta curta com possível dependência da pergunta. Na linha 25, a resposta informa que “não existe prazo previsto para a situação descrita na sua solicitação”. O gabarito é `c1`, mas a votação suave prevê `c5`. O enunciado é compreensível isoladamente, porém sua suficiência depende do que foi perguntado e da informação esperada. Essa é uma hipótese de dificuldade, sem prova de que o rótulo esteja incorreto ou de que o modelo tenha usado tal raciocínio.

Substituição indevida pelo GPT. Na confirmação RAG, o aumento de acertos em `c5` coexistiu com a perda de 21 acertos líquidos em `c234`. Esse padrão mostra por que revisar somente casos corrigidos produziria uma avaliação incompleta. A intervenção precisa compensar também os acertos que destrói; nesse estudo, o saldo não sustentou adoção.

Esses exemplos são ilustrativos, não uma estimativa da frequência de cada causa de erro. Os 453 erros do método final não podem ser atribuídos integralmente a anotações ambíguas ou à informação ausente.

9.2. Sobreposição e reutilização das avaliações

A auditoria identificou rótulos distintos para textos normalizados iguais em 316 grupos do treino, envolvendo 1.975 respostas, e em 11 grupos de Teste1, envolvendo 31 respostas. Há também 88 correspondências textuais exatas e 105 normalizadas entre Teste1 e o corpus original: 85 linhas normalizadas coincidem com o treinamento, 11 com a calibração e nove com a meta-validação. Todas as 900 respostas foram mantidas, sem exclusões ou correções orientadas pelo gabarito.

Logo, a separação por grupos dentro do desenvolvimento reduz um tipo de vazamento, mas não torna os arquivos oficiais inteiramente disjuntos nem assegura independência semântica. As repetições também limitam a hipótese de independência dos pares do teste de McNemar. Seus p-valores são apresentados com caráter descritivo e sem ignorar essas condições.

Teste1 e o conjunto reservado foram consultados em momentos anteriores do projeto. Congelar uma rodada antes de abrir novamente o gabarito protege aquela execução contra reajustes posteriores, mas não elimina a exposição histórica. A escolha principal após a trajetória comparativa é retrospectiva; não constitui uma nova estimativa confirmatória em dados nunca examinados.

10. Código, ambiente e reprodução

10.1. Repositórios e dependências

O código do EP2 está em github.com/felipe-txrod/pln-ep2. O EP1 utilizado como referência está em github.com/gabrielmarchesano/pln-ep1. Os comandos abaixo pressupõem a raiz do EP2. As avaliações utilizam diretórios novos para preservar resultados anteriores; o executor da entrega substitui seus próprios resultados a cada geração.

O ambiente de entrega utiliza Python 3.12 e execução em CPU por padrão. Para reprodução lexical, as versões fixadas são NumPy 2.3.3, SciPy 1.16.2, scikit-learn 1.7.2, joblib 1.5.2, threadpoolctl 3.6.0 e openpyxl 3.1.5. A inferência neural utiliza PyTorch 2.8.0, Transformers 4.57.1, PEFT 0.17.1 e Accelerate 1.10.1. O perfil padrão instala a distribuição CPU do PyTorch. Os experimentos em GPU utilizaram ROCm 6.4.1 na RX 6650 XT, disponível como perfil opcional de reprodução; o SVM utiliza CPU.

```
bash scripts/ensemble/setup-local.sh
bash scripts/ensemble/deliver-test2.sh --check
```

O primeiro comando cria `.venv-delivery/` com PyTorch CPU e as dependências lexicais e neurais. `--with-rocm` prepara o perfil opcional de GPU; `--with-neural` acrescenta as bibliotecas neurais preservando um PyTorch já instalado no ambiente selecionado, e `--minimal` instala somente a base lexical e de exportação. As dependências estão separadas em `requirements-delivery.txt` e `requirements-delivery-neural.txt`. A execução é nativa e dispensa Docker. O segundo comando verifica dependências, modelos e formato da entrada em CPU, sem realizar inferência. Os adaptadores e classificadores finais estão em `models/`; as bases públicas completas, planilhas oficiais e caches não são distribuídos integralmente pelo Git. A restauração da base NorBERTo é separada e verifica a revisão e os hashes fixados:

```
.venv-delivery/bin/python scripts/models/prepare-base.py
```

Esse comando pode baixar os arquivos públicos ausentes. O modelo-base é `Itau-Unibanco/NorBERTo-base`, revisão `db73446f89c96044863ea05a39f680524b84bccb`.

10.2. Aplicar a solução final sem novo treinamento

Para reproduzir as previsões em Teste1 com o modelo congelado de três candidatos e dois contribuintes efetivos:

```
.venv-delivery/bin/python -m ensemble_classifier predict-soft \
  --model models/ensemble-soft/model.json \
  --input data/test1.xlsx \
  --live-norberto --device cpu \
  --output-dir results/report-soft-test1-new
```

`--live-norberto` executa o adaptador local e dispensa os CSVs históricos do EP1. Para a opção GPU, prepara-se o perfil `--with-rocm` e seleciona-se `--device gpu`; na RX 6650 XT, utiliza-se `HSA_OVERRIDE_GFX_VERSION=10.3.0`, como no executor de entrega. Não há ajuste de pesos, calibradores ou modelos nesse comando.

Depois de persistir as previsões, a avaliação e a comparação com a entrega real do EP1 são separadas:

```
.venv-delivery/bin/python -m ensemble_classifier evaluate-soft \
  --run results/report-soft-test1-new \
  --gold data/test1-rotulado-para-teste-estatistico.xlsx \
  --output-dir results/report-soft-evaluation-new

.venv-delivery/bin/python scripts/ensemble/compare-ep1.py \
  --run results/report-soft-test1-new \
  --baseline ../EP1/deliveries/test1.xlsx \
  --output-dir results/report-soft-vs-ep1-new
```

O segundo comando requer a planilha entregue no EP1 nesse caminho. Recalcular a partir de previsões salvas dispensa nova inferência; a reprodução com GPU ou outro runtime pode apresentar pequenas diferenças numéricas em casos de fronteira.

10.3. Entrega oficial e evidências de desenvolvimento

A aplicação a Teste2 utiliza a mesma configuração congelada, sem consultar rótulos ou refazer seleção. Para gerar as previsões e preencher a planilha oficial:

```
bash scripts/ensemble/deliver-test2.sh
```

O executor resolve o caminho do projeto, verifica o ambiente e aplica a mistura congelada, utilizando CPU por padrão em todos os modelos. A opção explícita `--device gpu` transfere somente o NorBERTo para a GPU, quando o ambiente está preparado para ROCm. As previsões são salvas em `results/soft-voting-test2-final/` e a planilha de entrega em `deliveries/test2-soft-voting.xlsx`. A exportação valida as 897 previsões, suas classes, textos e posições e preenche somente a coluna de rótulos da planilha original. O restante do arquivo é preservado. Não se impõem cotas de classe nem se substituem rótulos manualmente.

Cada execução recalcula todos os escores dos membros, sem reutilizar previsões em cache, e substitui os resultados e o Excel após validação. Para gerar novamente apenas a planilha a partir de previsões concluídas e verificadas, sem nova inferência:

```
bash scripts/ensemble/deliver-test2.sh --export-only
```

A reprodução integral do ajuste da votação suave requer as partições e escores históricos identificados nas configurações, ou a inferência das bases locais, conforme `docs/guides/ensemble-soft.md`. Os passos das buscas e estudos alternativos estão nos guias de SVM, stacking, Qwen e RAG. Os agregados em `docs/results/data/` permitem conferir as conclusões sem repetir treinamentos ou chamadas remotas. A análise interpretável pode ser reproduzida com:

```
.venv-delivery/bin/python scripts/report/explain.py \
  --output results/report-interpretability-new.json
```

Esse script requer também as previsões A preservadas na finalização do SVM.

11. Considerações finais

O EP2 investigou diferentes formas de explorar complementaridade entre modelos, desde votos de classe até combinações aprendidas e intervenção seletiva de um modelo de linguagem. O resultado central é que maior capacidade e maior complexidade de combinação não produziram ganho consistente: maioria, stacking, Qwen, GPT seletivo e RAG não justificaram substituir a votação suave selecionada. A busca final do SVM também não demonstrou benefício suficiente para promover um novo componente.

Escolhemos, portanto, SVM original 75% e NorBERTo original 25%, com probabilidades calibradas. A combinação mantém o componente contextual do EP1, explora informação lexical forte e dispensa chamadas remotas. Em Teste1, alcançou 49,67% de acurácia e 49,58% de F1-macro, com 15 acertos adicionais frente à entrega do EP1. O ganho é descritivo, sem significância pelo teste de McNemar, e deve ser interpretado junto às limitações de contexto, anotação e reutilização dos conjuntos de avaliação.

Fontes experimentais e documentos de referência

Os documentos experimentais reúnem os agregados numéricos e os protocolos utilizados. Os caminhos abaixo são relativos à raiz do repositório EP2.

Fonte	Documento ou evidência
Enunciado e estrutura exigida	`docs/reference/assignment.md`; `docs/reference/report-template.md`
Reprodução da entrega nativa	`docs/guides/final-delivery.md`; `requirements-delivery.txt`
Maioria e votação suave	`docs/results/ensemble-majority.md`; `docs/results/ensemble-soft.md`
Comparação efetiva com EP1	`docs/results/data/ensemble-soft-vs-ep1.json`
Stacking e Qwen	`docs/results/ensemble-stacking.md`; `docs/results/ensemble-qwen.md`; `docs/results/qwen-ep1-model.md`
GPT inicial e seletivo	`docs/archive/gpt/first-result.md`; `docs/results/stacking-gpt-cascade.md`
RAG e confirmação	`docs/results/rag-cascade-pilot.md`; `docs/results/rag-cascade-confirmation.md`
Pesquisa offline de recuperação	`docs/results/rag-offline-research.md`
Busca e decisão final do SVM	`docs/results/svm-budgeted-optimization.md`; `docs/results/svm-final-optimization-final.md`
Interpretação lexical do relatório	`docs/report/data/interpretability.json`
Decisão da estratégia principal	`docs/decisions/ensemble-strategy.md`